



Linux
Professional
Institute

DEVOPS
TOOLS ENGINEER

TOPIC-TEN

Basics of puppet



Arash Foroughi

[linkedin.com/in/arashforoughi/](https://www.linkedin.com/in/arashforoughi/)



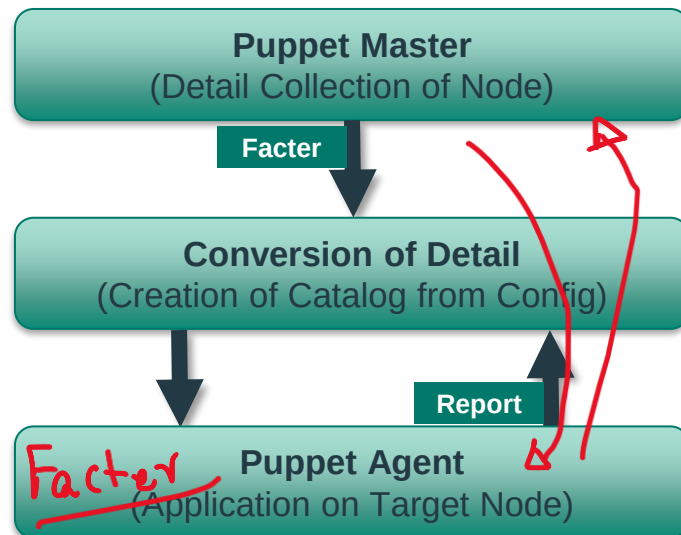
Puppet Overview & Architecture



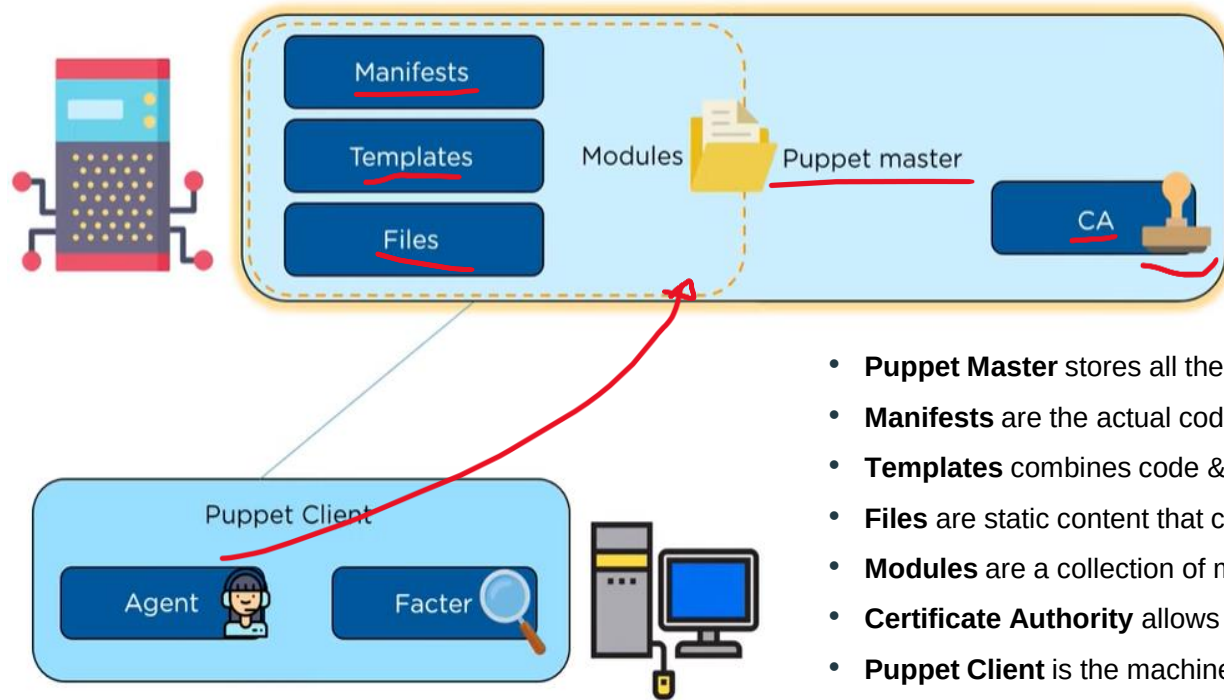
Puppet Overview

Server / client
Agent-Based

- Puppet is a **configuration management** technology to manage infra on physical or VMs.
 - It is **open-source** tool which developed using **Ruby**.
 - In Puppet There are one **Puppet Master** & some **Slave** nodes.
- 1) First, Master collect details of nodes using **facter** which there is on all nodes (like **Ohai** in **Chef**).
 - 2) Then, Master **compares** their config with defined configs & it creates a catalog to send to agents.
 - 3) Puppet Agent then applies configurations to get system into a **Desired State**.
 - 4) Finally, it sends a **report back** to master which helps to compare **current state** with **Catalog**.



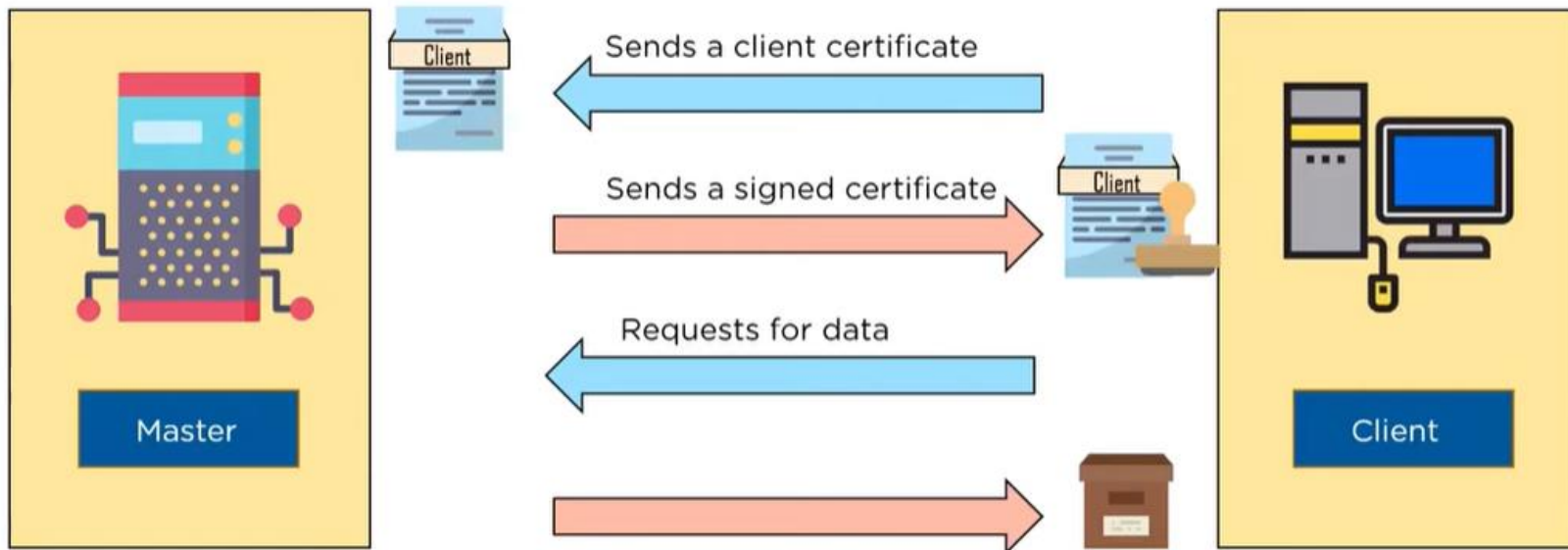
Puppet Key Components



- **Puppet Master** stores all the configuration files.
- **Manifests** are the actual codes for configuring the clients.
- **Templates** combines code & data to render final document.
- **Files** are static content that can be downloaded by clients.
- **Modules** are a collection of manifests, templates & files.
- **Certificate Authority** allows master to sign certificates sent by client.
- **Puppet Client** is the machine that requires to be configured.
- **Agent** continuously interacts with Master and sends certificates.
- **Facter** collects the current state of the clients.

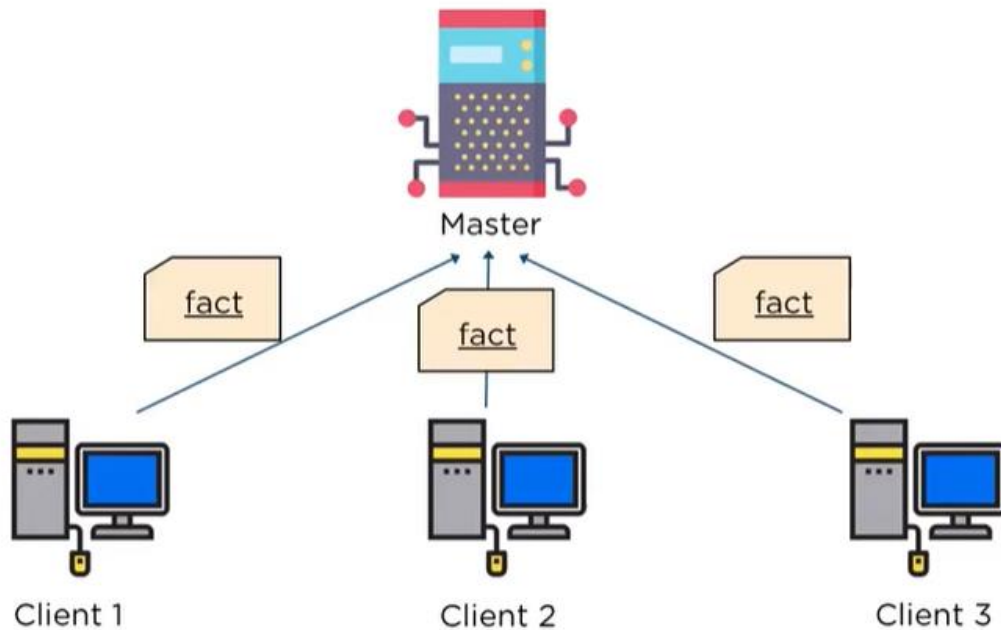
1. Puppet Architecture

- **SSL Certification:**



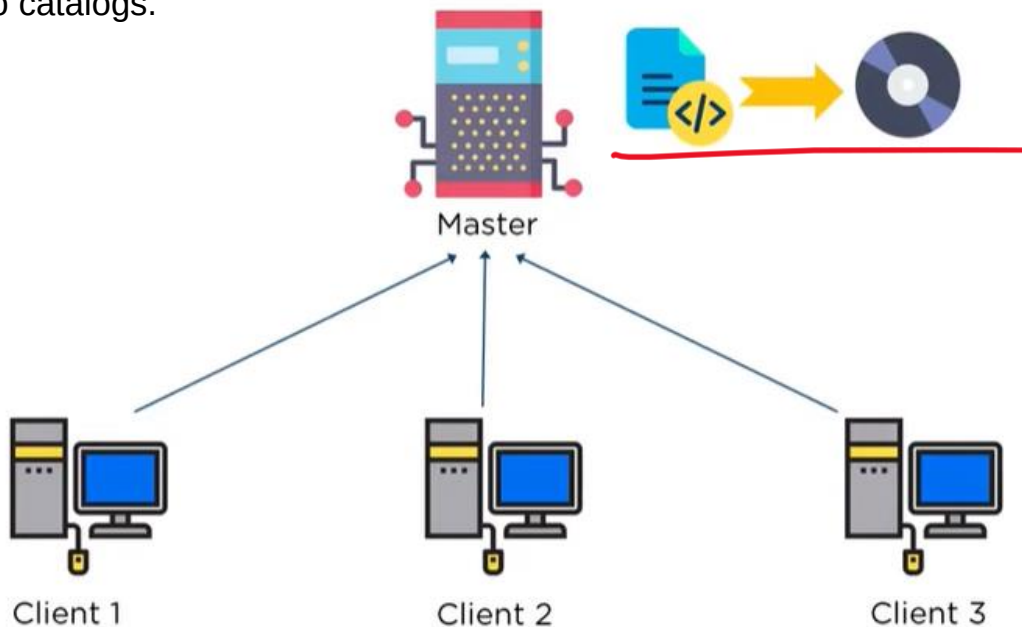
2. Puppet Architecture

- After SSL certification, the facter collects the state of clients and send it to master:



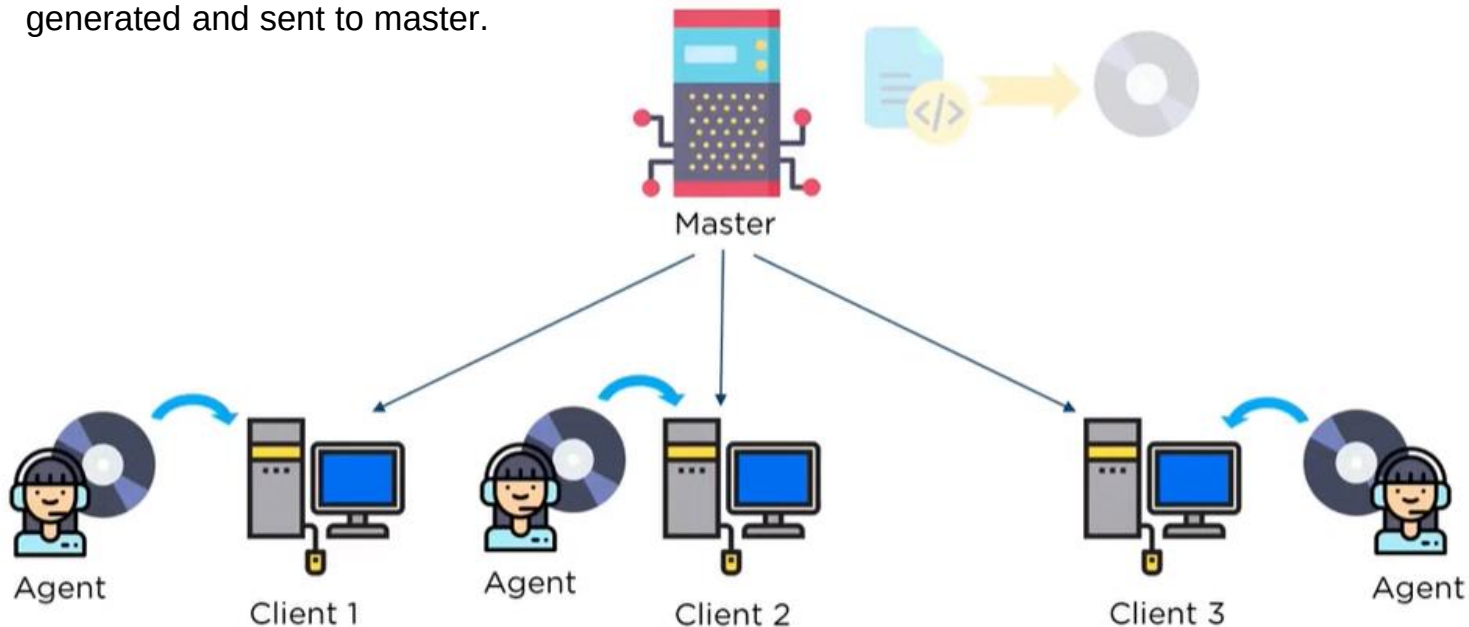
3. Puppet Architecture

- Based on the facts sent, master compare the facts with manifests and then compile the manifests into catalogs.



4. Puppet Architecture

- Finally, catalogs are sent on the clients & agent executes them on it's machine and a report will generated and sent to master.





Puppet Installation



Puppet Installation



Puppet Master – Ubuntu 20.04

1. Install Puppet Server
2. Start Puppet Server Service
6. Sign the Certificate
7. Write the Manifest



Puppet Client – Ubuntu 20.04

3. Install Puppet Agent
4. Start Puppet Agent Service
5. Create SSL Certificate
8. Execute Manifest/Catalog

Install Puppet Server on Master Node

- Before installing Puppet, needs to set up a unique hostname on **each nodes**.

```
>-----Install Puppet Server on Master Node-----<
root@master:~# hostnamectl set-hostname puppet
root@puppet:~# wget https://apt.puppetlabs.com/puppet7-release-focal.deb
root@puppet:~# dpkg -i puppet7-release-focal.deb
root@puppet:~# apt update; apt install puppetserver -y
root@puppet:~# vi /etc/default/puppetserver    --> Change JAVA_ARGS to 1GB
root@puppet:~# systemctl enable puppetserver; systemctl start puppetserver
```

Install Puppet Agent on Client Node

- Here is the installation of Puppet Agent on Client Nodes:

```
>-----Install Puppet Agent on Client Node-----<
root@client:~# wget https://apt.puppetlabs.com/puppet7-release-focal.deb
root@client:~# dpkg -i puppet7-release-focal.deb
root@client:~# apt update; apt install puppet-agent -y
root@client:~# /opt/puppetlabs/bin/puppet resource service puppet
                ensure=running enable=true
root@client:~# systemctl status puppet
```

Integrate Puppet Agent to Master Node

- For introducing puppet agents to master node, can use below procedure:

```
root@client:~# source /etc/profile.d/puppet-agent.sh
root@client:~# export PATH=/opt/puppetlabs/bin:$PATH
root@client:~# puppet config set server puppet --section main
root@client:~# cat /etc/puppetlabs/puppet/puppet.conf

root@client:~# puppet ssl bootstrap
root@puppet:~# puppetserver ca list
root@puppet:~# puppetserver ca sign --certname <name>
root@client:~# puppet ssl bootstrap
```

Testing Connectivity

- Here is an example for Testing Connectivity of agent with master:

```
root@puppet:~# touch /etc/puppetlabs/code/environments/production/manifests/sample.pp
root@client:~# /opt/puppetlabs/bin/puppet agent --test

Info: Using environment 'production'
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for client
Info: Applying configuration version '1646834312'
Notice: Applied catalog in 0.01 seconds
```

Applying First Catalog

- Here is an example for applying a catalog to clients:

```
root@puppet:~# vi /etc/puppetlabs/code/environments/production/manifests/sample.pp
{
  node 'ubuntu-4' {
    package{ 'apache2' :
      ensure=> installed,
    }
  }
}

root@client:~# /opt/puppetlabs/bin/puppet agent --test
Info: Using environment 'production'
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for ubuntu-4
Info: Applying configuration version '1646835255'
Notice: /Stage[main]/Main/Node[ubuntu-4]/Package[apache2]/ensure: created (corrective)
Notice: Applied catalog in 18.78 seconds
```

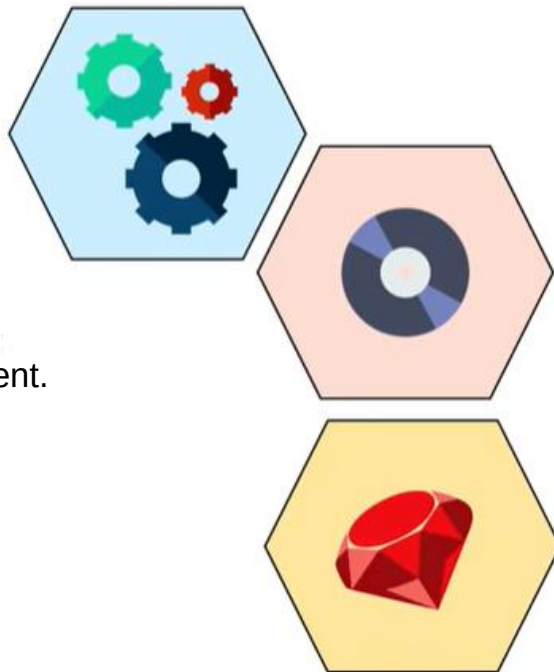


Manifests in Puppet



Manifests in Puppet

- Files written for configuring a node are called manifests.
- They are compiled into catalogs which are executed at the client.
- They are written in **Ruby** format with a **.pp** extension.



Manifests in Puppet

CREATE: Manifests are written by the system admin.

COMPILE: Manifests are compiled into catalogs.

DEPLOY: Catalogs are deployed on the clients.

EXECUTE: Catalogs are run on the client by the agent.

END: Clients are configured to desired state.

>-----Syntax of Manifests-----<

```
Resourcetype{ 'title' :  
  
  Attribute_name1 => attr_value  
  
  Attribute_name2 => attr_value  
  
}
```

Manifests in Puppet

- **Resource type** can be package, file, service,
- **Attribute_name** can be IP, ensure,
- **Attribute_value** can be present, start, absent,
- A manifest can be contained multiple Resourcetypes.

<https://forge.puppet.com/>

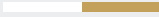
<https://www.unixmen.com/puppet-manifest-examples/>

```
root@puppet:~# vi sample1.pp
node default {      --> default keyword applies manifest to all clients.

    package {"apache2" :
    ensure => installed
    }

    → service {"apache2" :
    ensure => running
    }

    }
root@puppet:~# puppet apply -l /tmp/manifest.log sample1.pp
```



Thank you.

